CS260 | Data Structures
Written Assignment 1

Srinarayan Srikanth ss5837

1 Questions

Question 1 : 35 points

For each of the following code snippets:

- Give a tight bound on the runtime of the snippet, and
- briefly explain your bound.

If the runtime is asymptotically different in different cases, give a separate bound for each case and briefly explain what the cases are, e.g. "insertion sort runs in $\Theta(n)$ time in the best case, when the list is already sorted, because...".

(a) [7 points] `foo`

```
/* L is array of n integers */
function foo(L, n)
{
  a = 0
  while (a < n) {
    b = a
    x = 0
    while (b < n) {
      x = x + L[b]
      b++
    }
    L[a] = x
    a++
  }
  return L
}
```

Solution: Bound: $\Theta(n^2)$.

The outer `while` loop for a runs from 0 to $n-1$. For each value of a, the inner `while` loop for b starts at a and runs up to $n-1$, so it executes $n-a$ times. The total number of inner-loop iterations is

$$\sum_{a=0}^{n-1}(n-a) = \frac{n(n+1)}{2},$$

which is $\Theta(n^2)$.

(b) [7 points] `baz`

```
/* L is a list of n integers */
function baz(L, n) {
  for (i = 0; i < n; i++) {
    if (L[i] == 7) {
      return true
    }
  }
  return false
}
```

Solution: Best case: $\Theta(1)$, which occurs when $L[0] = 7$, so the function returns `true` immediately on the first iteration.

Worst case: $\Theta(n)$, which occurs when 7 does not appear in L (or appears only at $L[n-1]$), so the loop must run through all n elements before returning.

(c) [7 points] `bar`

```
/* a, b, c, d are all integers */
function bar(a, b, c, d) {
  if (a < b) {
    if (c < d) {
      if (a < c) {
        return a
      } else {
        return c
      }
    } else {
      if (a < d) {
        return a
      } else {
        return d
      }
    }
  } else {
    if (c < d) {
      if (b < c) {
        return b
      } else {
        return c
      }
    } else {
      if (b < d) {
        return b
```

```
      } else {
        return d
      }
    }
  }
}
```

Solution: Bound: $\Theta(1)$.
The function is a fixed sequence of nested `if-else` comparisons with no loops or recursion. Every path through the function performs exactly three comparisons before returning, which is constant work independent of any input size.

(d) [7 points] `qux`

```
/* n is a positive integer */
function qux(n) {
  for (i = 0; i < n; i++) {
    for (j = 0; j < n; j++) {
      for (k = 0; k < n; k++) {
        for (l = 0; l < 8; l++) {
          print(i * (j + k) - l)
        }
      }
    }
  }
}
```

Solution: Bound: $\Theta(n^3)$.
The three outer loops (i, j, k) each run exactly n times, and the innermost loop (l) always runs exactly 8 times (a constant), independent of n. The total number of `print` calls is $8n^3$, which is $\Theta(n^3)$.

(e) [7 points] `xyzzy`

```
/* n is a positive integer */
function xyzzy(n) {
  x = 2
  for (i = 0; i < n; i++) {
    x = x * 7
  }
  for (j = 0; j < 2*n; j++) {
    x = x / 3
  }
  return x
}
```

Solution: Bound: $\Theta(n)$.

The two `for` loops are sequential, not nested. The first runs n times and the second runs $2n$ times, each doing constant work per iteration. The total is $n+2n = 3n$, which is $\Theta(n)$.

Question 2 : 20 points

Order the following functions from least to greatest growth rate, such that for any two functions f and g, if f comes before g, then $f(n) = O(g(n))$:

- $f_1(n) = n^n$
- $f_2(n) = n^{0.00001}$
- $f_3(n) = n^2$
- $f_4(n) = \lg(n)$
- $f_5(n) = 2^n$

You must prove each ranking, but you may assume the transitive property is true, which says that if $f(n) = O(g(n))$ and $g(n) = O(h(n))$, then $f(n) = O(h(n))$.

This means that if the functions were f, g, h and you gave the order f, g, h, then you'd only need to prove that $f(n) = O(g(n))$ and $g(n) = O(h(n))$, as you may assume that this implies $f(n) = O(h(n))$.

You must fully simplify your limit ratios to receive credit. If you haven't simplified your limit ratio to 0, ∞, or a positive constant, you're not finished simplifying.

Solution: Order (least to greatest): $f_4(n),\ f_2(n),\ f_3(n),\ f_5(n),\ f_1(n)$

1. $f_4(n) = O(f_2(n))$**:**

$$\lim_{n\to\infty}\left(\frac{\lg(n)}{n^{0.00001}}\right) = \lim_{n\to\infty}\left(\frac{\ln(n)/\ln 2}{n^{0.00001}}\right)$$

Applying L'Hôpital's rule (both numerator and denominator $\to \infty$):

$$= \lim_{n\to\infty}\left(\frac{1/(n\ln 2)}{0.00001\, n^{-0.99999}}\right) = \lim_{n\to\infty}\left(\frac{1}{0.00001\ln 2\cdot n^{0.00001}}\right) = 0.$$

So $f_4(n) = O(f_2(n))$.

2. $f_2(n) = O(f_3(n))$**:**

$$\lim_{n\to\infty}\left(\frac{n^{0.00001}}{n^2}\right) = \lim_{n\to\infty}\left(n^{-1.99999}\right) = 0.$$

So $f_2(n) = O(f_3(n))$.

3. $f_3(n) = O(f_5(n))$**:**

$$\lim_{n\to\infty}\left(\frac{n^2}{2^n}\right)$$

Applying L'Hôpital's rule twice:

$$= \lim_{n\to\infty}\left(\frac{2n}{2^n \ln 2}\right) = \lim_{n\to\infty}\left(\frac{2}{2^n (\ln 2)^2}\right) = 0.$$

So $f_3(n) = O(f_5(n))$.

4. $f_5(n) = O(f_1(n))$**:**

$$\lim_{n\to\infty}\left(\frac{2^n}{n^n}\right) = \lim_{n\to\infty}\left(\left(\frac{2}{n}\right)^n\right) = 0,$$

since $2/n \to 0$ and is raised to an ever-growing power. So $f_5(n) = O(f_1(n))$.

By transitivity, $f_4(n) = O(f_2(n)) = O(f_3(n)) = O(f_5(n)) = O(f_1(n))$, giving the order above.

Question 3 : 15 points

Consider the following function `compute`:

```
1  /* n is a positive integer */
2  function compute(n)
3  {
4    if (n == 0)
5    {
6      return 0
7    }
8    x = 0
9    for (i = 1; i <= n; i++)
10   {
11     x = x + i
12   }
13   return compute(n - 1) + x
14 }
```

(a) [5 points] Give a recurrence that represents the runtime of `compute`.

Solution: The `for` loop does $\Theta(n)$ work, followed by one recursive call on $n-1$:

$$T(n) = T(n-1) + \Theta(n), \qquad T(0) = \Theta(1).$$

(b) [10 points] Solve your recurrence using the substitution method, then give a tight bound on the runtime of `compute`.

Solution: Write $T(n) = T(n-1) + n$. Expanding:

$$T(n) = T(n-1) + n = T(n-2) + (n-1) + n = T(n-3) + (n-2) + (n-1) + n = \cdots$$

After k substitutions:

$$T(n) = T(n-k) + \sum_{i=0}^{k-1}(n-i).$$

The base case occurs when $n - k = 0$, i.e. $k = n$:

$$T(n) = T(0) + \sum_{i=0}^{n-1}(n-i) = T(0) + \sum_{j=1}^{n} j = \Theta(1) + \frac{n(n+1)}{2}.$$

Tight bound: $T(n) = \Theta(n^2)$.

Question 4 : 30 points

Professor Holmes has come up with a new sorting algorithm. He calls it **Trimerge Sort** and "claims" that it is asymptotically faster than **Merge Sort**, despite the fact both the algorithms operate using similar logic. But, unlike Merge Sort, Trimerge Sort splits the input array into *(roughly)* 3 equal parts at each step of the recursion as long as the array is splittable *(i.e., has at least 3 elements)*. Trimerge Sort's merge subroutine, similar in principle to the one used by Merge Sort, takes 3 sorted subarrays and merges them to produce a single sorted array. Given all of this, answer the following questions.

(a) [5 points] In Merge Sort, the **merge subroutine** makes n comparisons to merge 2 arrays of size $n/2$, which takes $\Theta(n)$ time. How many comparisons (in exact numbers) will the merge subroutine of Trimerge Sort make to merge 3 arrays of size $n/3$? What would be the $\Theta(\cdots)$ bound on the running time for merge subroutine?

Solution: To place each of the n elements into the merged output, the subroutine must find the minimum of the current front elements of the 3 subarrays. Finding the minimum of 3 elements takes 2 comparisons (compare the first two, then compare that winner against the third). Doing this for each of the n elements placed gives

$$2n \text{ comparisons},$$

matching the same style of count given for the 2-array case (n comparisons for 2 arrays $\Rightarrow 2n$ for 3 arrays, since one extra comparison is needed per element to pick among 3 candidates instead of 2).

Bound: $\Theta(n)$.

(b) [5 points] What is the recurrence relation for the Trimerge Sort algorithm? Also explain it in plain English what each term in the recurrence relation stands for.

Solution:

$$T(n) = 3T(n/3) + \Theta(n)$$

- $T(n)$: total time to sort an array of size n with Trimerge Sort.
- $3T(n/3)$: the cost of recursively sorting the 3 subarrays, each of (roughly) size $n/3$.

- $\Theta(n)$: the cost of the merge subroutine that combines the 3 sorted subarrays back into one sorted array of size n.

(c) [8 points] Prove the $\Theta(\cdots)$ bound on the running time of the Trimerge Sort algorithm using the **Recursion Tree Method** covered in class.

Solution: Let the merge cost at a node handling m elements be cm.

- **Level 0 (root):** 1 node of size n, cost cn.
- **Level 1:** 3 nodes of size $n/3$, total cost $3 \cdot c(n/3) = cn$.
- **Level i:** 3^i nodes, each of size $n/3^i$, total cost $3^i \cdot c(n/3^i) = cn$.
- **Height:** the array shrinks by a factor of 3 each level until reaching size 1, so the tree has $\log_3 n$ levels.

Every level costs cn, and there are $\log_3 n$ levels, so the total work is

$$\sum_{i=0}^{\log_3 n} cn = cn \log_3 n.$$

Tight bound: $\Theta(n \log n)$.

(d) [8 points] Prove the $\Theta(\cdots)$ bound on the running time of the Trimerge Sort algorithm using **Equation Expansion** method over the recurrence relation you came up with in part (b).

Solution: Starting from $T(n) = 3T(n/3) + cn$, expand repeatedly:

$$T(n) = 3\left(3T(n/9) + c\frac{n}{3}\right) + cn = 9T(n/9) + 2cn$$

$$T(n) = 9\left(3T(n/27) + c\frac{n}{9}\right) + 2cn = 27T(n/27) + 3cn$$

After k expansions:

$$T(n) = 3^k T(n/3^k) + kcn.$$

The recursion bottoms out when $n/3^k = 1$, i.e. $k = \log_3 n$. Substituting:

$$T(n) = 3^{\log_3 n} T(1) + (\log_3 n)\, cn = n \cdot \Theta(1) + cn \log_3 n.$$

Tight bound: $\Theta(n \log n)$.

(e) [4 points] Is Trimerge Sort asymptotically faster than Merge Sort? Why/why not? Reason through your answer using **a mathematical proo**f.
*(**Hint:** Use the limit method demonstrated in class as well as the recitation on Asymptotic Analysis & Recurrence Relations)*

Solution: Merge Sort's recurrence, $T_m(n) = 2T_m(n/2) + \Theta(n)$, solves to $\Theta(n \log_2 n)$. Trimerge Sort solves to $\Theta(n \log_3 n)$ (parts (c), (d)). Compare via the limit ratio:

$$\lim_{n\to\infty} \left(\frac{n \log_3 n}{n \log_2 n} \right) = \lim_{n\to\infty} \left(\frac{\ln n / \ln 3}{\ln n / \ln 2} \right) = \frac{\ln 2}{\ln 3} \approx 0.63,$$

a positive constant. Since the limit is a positive, finite constant, $n \log_3 n = \Theta(n \log_2 n)$, so the two functions have the *same* asymptotic growth rate.

Conclusion: Trimerge Sort is *not* asymptotically faster than Merge Sort; both run in $\Theta(n \log n)$ time.